Heap

23-08-23

↳ simulation

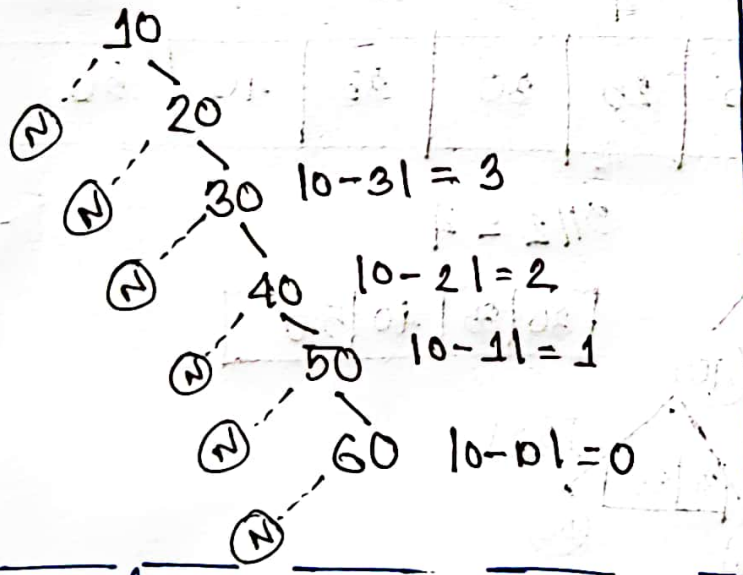
↳ drive video.

TREE

BST

1) You are given an unbalanced tree. Make the tree balanced.

Unbalanced



Step

1) inorder traversal & store in an array.

a = | 10 | 20 | 30 | 40 | 50 | 60 |

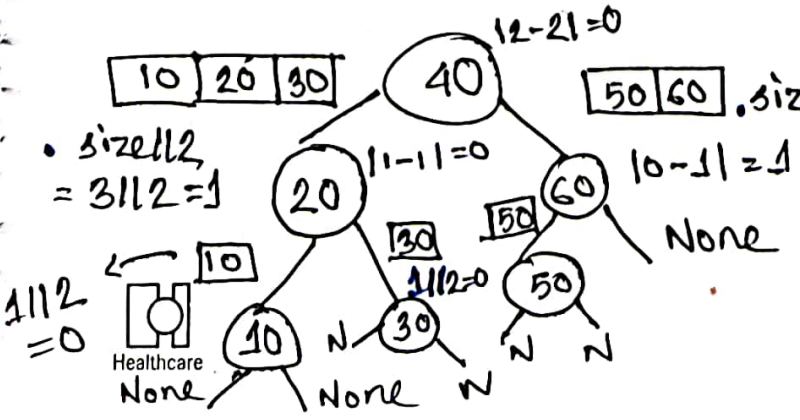
2) root ⇒ (size//2) index

left subtree ⇒ (0-size//2-1)

right subtree ⇒ (size//2+1, size)

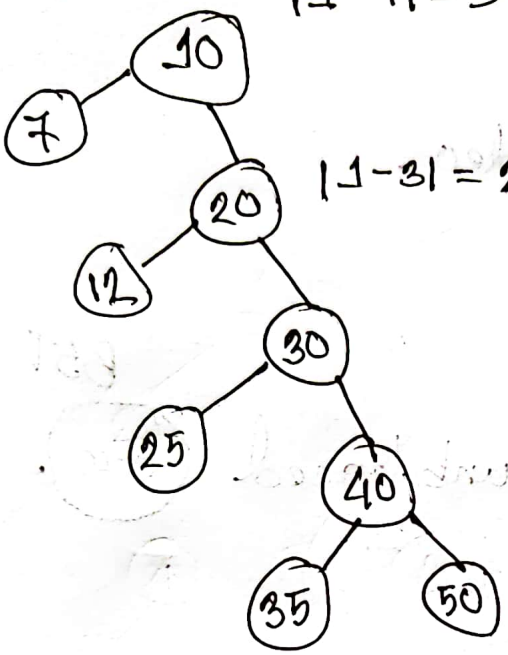
Balanced

• size//2 = 6//2 = 3



Emistat®
Ondansetron

CamScanner



$|1-4| = 3$

$|1-3| = 2$

∴ It's unbalanced

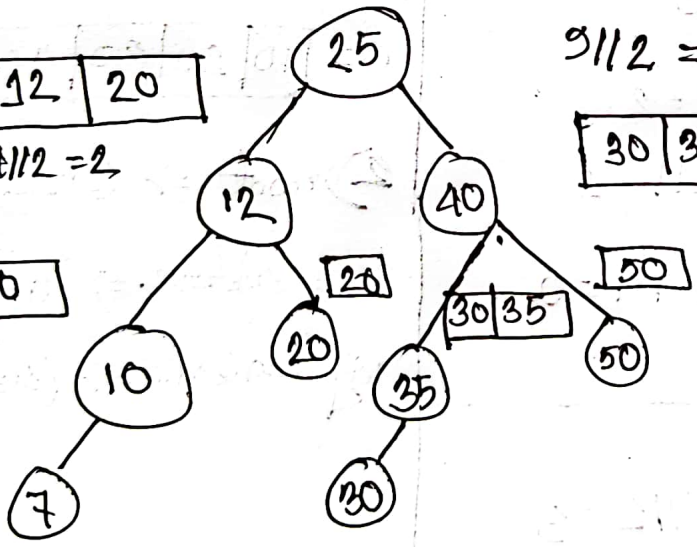
0	1	2	3	4	5	6	7	8
7	10	12	20	25	30	35	40	50

array =

7	10	12	20

4/2 = 2

7	10



9/2 = 4

30	35	40	50

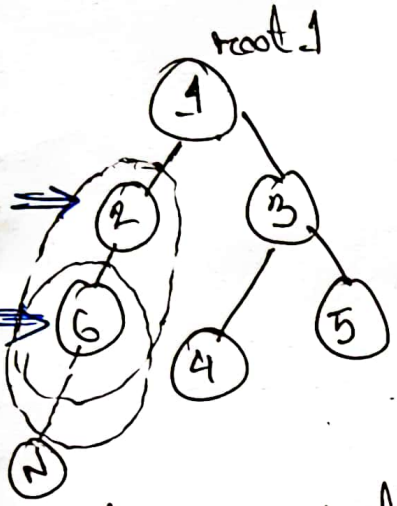
20

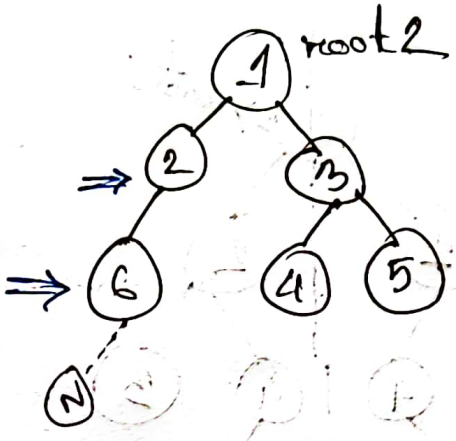
30	35

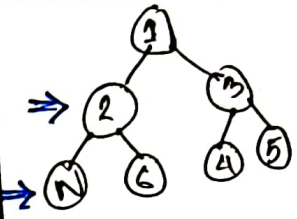
50

Ques

You are give two binary trees. Check whether the trees are equivalent.







will check none.elem error

```
def equivalent (root1, root2):
    (3) if root1.elem != root2.elem:
            return False
    (2) if root1 == None and root2 == None:
            return True
    (4) if root1.elem == root2.elem:
            rtn equivalent (root1.left, root2.left) and
                equivalent (root1.right, root2.right)
```

1st condition

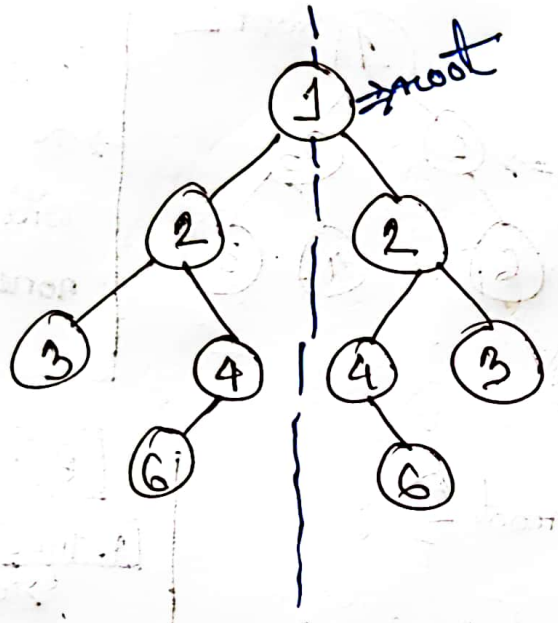
```
if (root1 == None and root2 != None) or
   (root1 != None and root2 == None):
    return False.
```

Base Case

1. True
   root1 & root2 = None
2. False
   root1.elem != root2.elem

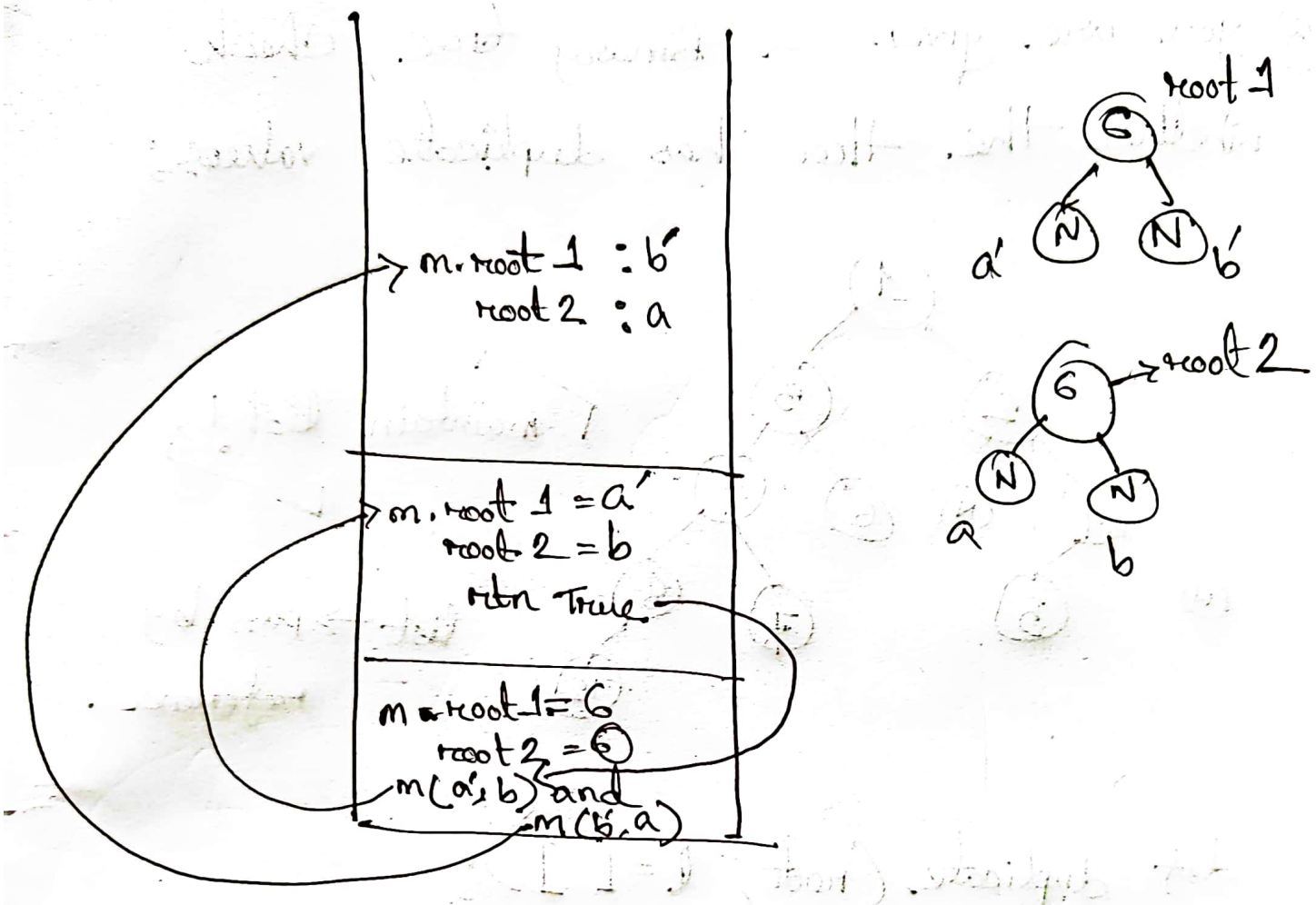
Healthcare
Emistat® Ondansetron
CamScanner

2) You are given a Binary Tree. Check whether the trees are symmetric/mirror:




```
def mirror (root_1, root_2):
    1)
    2)
    3)
    4) if root_1.elem == root_2.elem:
        return mirror (root_1.left, root_2.right) and
               mirror (root_1.right, root_2.left)

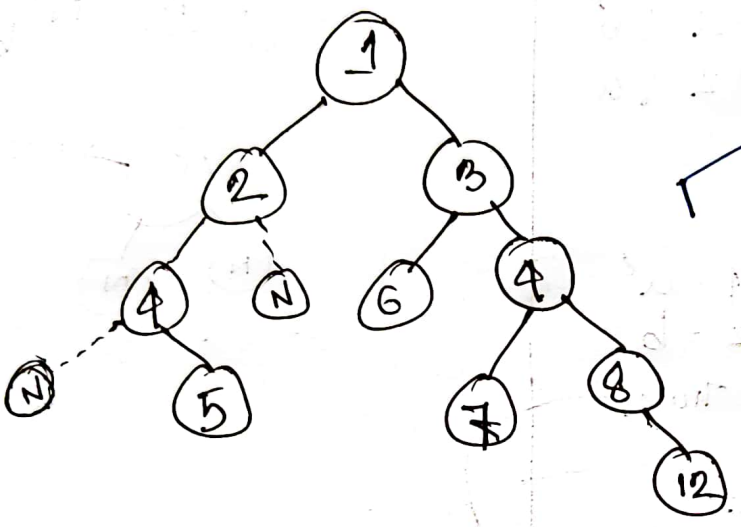
print (mirror (root, root))
```



Emistat®

CamScanner

3) you are given a Binary tree, check whether the tree has duplicate values:



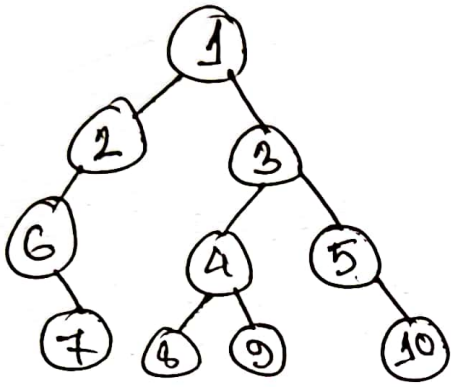
maintain list!

list → pass by reference.

```
def duplicate(root, l=[]):
    if root == None:
        returns False

    if root.elem in l:
        rtn ~~False~~ True
    else:
        l.append(root.elem)
    rtn duplicate(root.left, l) or
        duplicate(root.right, l)
```

* You are given a Binary tree. Print all the Nodes that are at k^{th} distance from root.



k=2

6 4 5

```
def distance_root (root, k, counter=0):
    if root == None:
        retn None
    if counter == k:
        print (root.elem)
    else:
        distance_root (root.left, k, c+1)
        distance_root (root.right, k, c+1)
```

Healthcare

Emistat®
Ondansetron

CamScanner